SafetyMonitor 코드베이스 종합 Audit 보고서

06.safetymonitor · 9차원 정적 분석 · 2026-06-12

방식 — 15개 bounded 파일그룹(담당 파일만 읽는 스캔 에이전트) → 그룹별 **적대적 검증**(인용 코드 재확인). 30 에이전트 · 358 도구호출. 적대적 검증이 **false positive 3건 제거**.
스코프 — apps/dashboard-web(Next.js) · backend/infrastructure(AWS CDK) · apps/worker-phone(Flutter) · apps/worker-watch-poc(watchOS Swift) · packages(analyzers·schemas) · scripts. 제외: node_modules·cdk.out·.next·build.
제약 — 파일 수정 0(읽기 전용 보고). 모든 발견은 실제 코드 인용. 단순 스타일 제외.

1. Executive Summary

CRITICAL	HIGH	MEDIUM	LOW	합계
0	1	10	49	60

차원별: 타입 안전성 1 · 보안 5 · 정확성·로직 23 · 성능 10 · 테스트 6 · 의존성·설정 2 · Dead code 8 · 문서·일관성 5 · 아키텍처 0(클린)

CRITICAL 0의 의미 — 결함 부재가 아니라 프로토타입 스코프(세션마다 회전하는 임시 cloudflared 터널·합성/직원 데이터·dev localhost 바인딩) 덕분. 적대적 검증이 보안 발견 2건을 '실서비스면 HIGH'에서 프로토타입 맥락으로 MEDIUM 강등. 각 발견의 '왜 문제'에 상용화 시 위험을 명시. **아키텍처 0건** = analyzers 순수성·계층분리·순환의존 무위반(긍정 신호).

가장 위험한 5건

1. HIGH **FallDetector 공유 가변 상태가 백그라운드 큐와 MainActor 간 동기화 없이 경쟁**
   `apps/worker-watch-poc/Sources/FallDetector.swift:65-86, 88-130, 147-155`
2. MEDIUM **AED 점검 기록 POST 무인증 — 생명안전 장비 정비주기 위조 가능**
   `apps/dashboard-web/app/api/aed/inspect/route.ts:7-41`
3. MEDIUM **CCTV 프록시가 경로 검증 없는 무인증 오픈 프록시(경로 트래버설·임의 메서드 포워딩)**
   `apps/dashboard-web/app/ai-cctv/[...path]/route.ts:10-19`
4. MEDIUM **vital alert 전이 판정이 workActivity(s) 활동 게이트에 의존해 동일 HR 이 활동 메트릭 유무로 위험→정상 토글 가능**
   `apps/dashboard-web/app/api/mock/store.ts:33-40, 86-93`
5. MEDIUM **KMA 기상특보 fetch 에 타임아웃 없음 — 업스트림 hang 시 요청 무한 대기**
   `apps/dashboard-web/app/lib/kma.ts:159`

2. 차원별 상세 발견사항 (severity 내림차순)

[1] 타입 안전성 — 1건

LOW [1] 타입 안전성 #12 · apps/dashboard-web/app/lib/kma.ts:174-177

KMA t6/t7 파싱이 items.item 형태 가정 — 비배열·단일객체 외 변형에 first 추출 취약

왜 문제: resultCode 가 "00" 인데도 `items` 가 빈 문자열/없음(일부 data.go.kr 응답 변형)이면 `first` 가 undefined 가 되고 `String(undefined?.t6 ?? "")` 로 빈 특보가 된다(크래시는 없음). 다만 typhoon.ts 의 `extractRows`/kosha.ts 의 tolerant 추출과 달리 여기는 `items.item` 단일 경로만 가정해, 응답 shape 가 조금만 달라도 라이브인데 특보를 놓친다(false-negative). 안전 관점에서 특보 누락은 정상 점등보다 위험.

수정 힌트: typhoon/kosha 와 동일하게 `items.item`/`items`(배열)/`item` 단일을 모두 수용하는 tolerant 추출 헬퍼로 통일.

[2] 보안 — 5건

MEDIUM [2] 보안 #2 · apps/dashboard-web/app/api/aed/inspect/route.ts:7-41

AED 점검 기록 POST 무인증 — 생명안전 장비 정비주기 위조 가능

왜 문제: 이 엔드포인트는 verifyBasicAuth 검증이 전혀 없다(주석 '무인증·PII 아님'). aedId 만 알면(레지스트리 AED-01~07 = 추측 가능, NFC URL 노출) 누구나 가짜 '배터리/패드 교체' 기록을 POST 할 수 있다. 교체 기록은 대시보드의 AED 교환주기(battCycleMonths/padCycleMonths)를 리셋해 만료 임박/경과 경고를 가린다 → 실제로 만료된 제세동기가 '정상'으로 표시되는 정비 무결성 위조. PII 노출은 없으나 쓰기(write) 경로에 인증·서명·rate limit 이 전무해 생명안전 데이터의 신뢰성이 깨진다. 공개 cloudflared 터널로 URL 이 노출되면 즉시 악용 가능. 상용화 시 NFC 태그에 HMAC 서명 또는 장비별 토큰이 필요하다.

```
export async function POST(request: Request) {
  let body: unknown;
  ...
  const device = findAedDevice(aedId);
  ...
  recordAedInspection({ aedId, loc: device.loc, inspector, items, ts });
  return NextResponse.json({ ok: true, ... }, { status: 201, ... });
```

수정 힌트: AED 점검 POST 에 최소한의 장비별 비밀(NFC URL 에 서명 토큰 ?sig=HMAC(aedId)) 검증 또는 현장 관리자 인증을 추가하고, IP/장비별 rate limit 으로 무더기 위조 기록을 차단. 최소 audit 용으로 source/요청 출처를 함께 기록.

MEDIUM [2] 보안 #3 · apps/dashboard-web/app/ai-cctv/[...path]/route.ts:10-19

CCTV 프록시가 경로 검증 없는 무인증 오픈 프록시(경로 트래버설·임의 메서드 포워딩)

왜 문제: catch-all `path` 세그먼트와 쿼리스트링이 검증·allowlist 없이 ORIGIN 뒤에 그대로 결합된다. `/ai-cctv/..%2f..%2fsecret` 같은 트래버설로 ORIGIN 호스트의 의도치 않은 경로에 도달하거나, 인증 없이 GET/POST + body 를 발표 PC localhost:8000 에 임의 포워딩하는 오픈 프록시가 된다. 현재 ORIGIN 이 localhost 고정이라 외부 SSRF 표면은 제한적이지만, 공개 cloudflared 터널로 :3000 이 노출되면 외부에서 백엔드 제어 엔드포인트(POST /cctv/connection 등)를 인증 없이 호출할 수 있다(상용화 시 백엔드를 원격/사내망에 두면 본격 SSRF·인가 우회).

```
const target = `${ORIGIN}/${path.join("/")}${req.nextUrl.search}`;
  const init: RequestInit & { duplex?: "half" } = { method: req.method };
  if (req.method !== "GET" && req.method !== "HEAD") {
    const ct = req.headers.get("content-type");
    init.headers = ct ? { "content-type": ct } : undefined;
    init.body = await req.text();
  }
  ... const upstream = await fetch(target, init);
```

수정 힌트: 허용 경로 prefix(`cctv/stream*`, `cctv/stats*`, `cctv/connection*`)만 통과시키는 allowlist 추가 + path 세그먼트에 `..`·인코딩 트래버설 거부. 제어성 POST 엔드포인트는 별도 인증(세션 토큰) 게이트.

LOW [2] 보안 #13 · apps/dashboard-web/lib/admin-auth.ts:65-77

로그인 시도 카운터가 공격자 제어 id 로 키잉 — 정상 관리자 락아웃·Map DoS

왜 문제: 잠금 카운터가 클라이언트가 보낸 id 문자열로 키잉된다(login route 가 body.id 를 그대로 loginLockRemainingMs/recordLoginResult 에 전달). (1) 공격자가 실제 'admin' id 로 5회 실패시키면 정상 관리자가 5분간 로그인 불가(targeted lockout DoS). (2) 매 요청 임의의 distinct id 를 보내면 loginAttempts Map 이 무한 증가(in-memory 고갈). length 제한도 없다. 프로토타입에선 영향 제한적이나 공개 URL 공유 시 발표 도중 관리자 락아웃 가능.

수정 힌트: 잠금 키를 id 단독이 아닌 IP(또는 IP+id) 기준으로 두고, loginAttempts Map 에 항목 수 상한·만료 sweep 을 추가. id 길이도 검증.

LOW [2] 보안 #14 · apps/dashboard-web/lib/admin-auth.ts:111-131

verifyBasicAuth 가 Bearer 토큰 자리에서도 base64(id:pw) 자격을 복호·검증

왜 문제: 정규식이 basic·bearer 를 동일 취급하므로 'Bearer <base64(id:pw)>' 도 세션 검증 실패 후 자격 복호 경로로 빠진다. 즉 평문 자격(가역적 base64)이 Authorization: Bearer 헤더로도 통과한다. ADMIN_PASSWORD 미설정 시 fail-closed 라 실제 위험은 낮으나, 비밀번호 설정 운영에선 평문 자격이 Bearer 자리로 전송·로깅될 수 있어 의도(세션 토큰 전용)와 어긋난다. 하위호환 경로 자체가 평문 자격을 매 요청 전송하게 만든다.

수정 힌트: Bearer 는 세션 토큰만, Basic 만 base64 자격을 받도록 스킴별 분기. 하위호환 자격 경로는 발표 후 제거하고 세션 토큰으로 일원화.

LOW [2] 보안 #15 · apps/dashboard-web/app/components/LoginScreen.tsx:145

로그인 ID 입력 기본값을 'admin' 으로 하드코딩

왜 문제: 로그인 폼 ID 필드 초기값이 기본 관리자 계정명 'admin' 으로 미리 채워져 있어, 기본 자격증명(admin/safety2026)을 코드/UI 에 노출해 추측 공격 표면을 키운다. 프로토타입 편의이나 공개 cloudflared URL 공유 시 비밀번호만 맞히면 되는 상태가 된다. 인증 자체는 서버(/api/admin/login)에서 하므로 CRITICAL 은 아니지만 상용화 시 제거 대상.

수정 힌트: 기본값을 빈 문자열로 두고, 기본 관리자 자격증명도 env 강제(없으면 기동 실패)로 전환.

[3] 정확성·로직 — 23건

HIGH [3] 정확성·로직 #1 · apps/worker-watch-poc/Sources/FallDetector.swift:65-86, 88-130, 147-155

FallDetector 공유 가변 상태가 백그라운드 큐와 MainActor 간 동기화 없이 경쟁

왜 문제: process() 는 deviceMotion 콜백을 통해 OperationQueue(백그라운드 스레드)에서 impactAt/impactPeakG/lowActivityAccum/lastSampleAt/peakRotation/freefallPrecededImpact/lastFreefallAt/lastFallAt 인스턴스 변수를 읽고 쓴다. 그런데 stop()→resetEvent() 는 WorkoutManager(@MainActor)의 finishAndEnd(173행)·didFailWithError(209행)에서 호출되어 동일 변수들을 메인 스레드에서 동시 변형한다. FallDetector 는 actor/락이 없는 plain final class 라 두 스레드의 동시 접근은 Swift 메모리 모델상 데이터 레이스(미정의 동작)다. 낙상 디텍터 상태 손상은 안전 핵심 기능에서 오탐·미탐을 유발할 수 있다. 상용화 시 Swift 6 strict concurrency 로 컴파일 에러가 되며, 현재도 stopDeviceMotionUpdates 직후에도 이미 큐에 적재된 콜백이 resetEvent 와 동시 실행될 수 있다.

```
motion.startDeviceMotionUpdates(to: queue) { [weak self] dm, _ in ... self.process(totalG: totalG, rotMag: rotMag, at: Date())
}
...
func stop() {
    if motion.isDeviceMotionActive { motion.stopDeviceMotionUpdates() }
    resetEvent()
}
```

수정 힌트: FallDetector 의 가변 상태 접근을 단일 직렬 컨텍스트로 통일한다. 예: stop()/resetEvent() 도 같은 OperationQueue(또는 전용 DispatchQueue)에서 실행하도록 큐에 디스패치하거나, 디텍터를 actor 로 전환하거나, 상태 변형을 모두 직렬 큐로 격리한다.

MEDIUM [3] 정확성·로직 #4 · apps/dashboard-web/app/api/mock/store.ts:33-40, 86-93

vital alert 전이 판정이 workActivity(s) 활동 게이트에 의존해 동일 HR 이 활동 메트릭 유무로 위험→정상 토글 가능

왜 문제: vitalDanger 는 병합 샘플의 heartRate 와 workActivity(s) 로 위험을 판정하는데, 활동 입력(steps/activeEnergy 등)은 60초 sub-cadence 의 sparse·sticky 필드다. 같은 HR 값이라도 활동 메트릭이 병합에 섞이느냐에 따라 작업밴드(135/155)와 안정밴드(110/130) 사이를 오가며 danger 판정이 바뀐다. prevMerged(추가 직전)와 curMerged(추가 직후)의 활동 컨텍스트가 sticky carry-forward 타이밍 때문에 달라지면, 실제 HR 악화가 아니라 활동 게이트 전환만으로 false transition 이 1건 기록되거나(노이즈 알림) 반대로 진짜 악화가 게이트로 가려질 수 있다. alerts 는 산업재해 자료 첨부용이라 정확성이 중요하다.

```
function vitalDanger(s: VitalSample | undefined): boolean {
  if (!s) return false;
  return (
    heartRateStatus(s.heartRate, workActivity(s)) === "danger" || // 작업 중이면 작업부하 밴드
    ...
  );
}
  ...
      if (!vitalDanger(prevMerged) && vitalDanger(curMerged)) {
        recordAlert({ ... type: "vital", ts: curMerged?.ts ?? sample.ts });
```

수정 힌트: alert 전이 판정에서는 활동 게이트를 끄고 안정 임계(보수적·false-negative 0)로 vitalDanger 를 계산하거나, prev/cur 양쪽에서 동일한 활동 컨텍스트를 명시적으로 고정해 게이트 전환만으로 transition 이 발화하지 않도록 한다.

MEDIUM [3] 정확성·로직 #5 · apps/dashboard-web/app/lib/kma.ts:159

KMA 기상특보 fetch 에 타임아웃 없음 — 업스트림 hang 시 요청 무한 대기

왜 문제: weather route 는 AbortController 6초 타임아웃을 걸지만(`fetchWithTimeout`), 같은 data.go.kr 계열인 이 KMA 호출은 타임아웃이 전혀 없다. data.go.kr 게이트웨이가 응답을 끊지 않고 매달리면(흔한 장애 모드) `fetch`/`res.json()` 이 영원히 pending 상태가 되고 try/catch 는 reject 가 안 나서 잡지 못한다 → /api/kma-warning 요청과 그걸 호출하는 대시보드 폴링이 무한 대기. 10분 캐시도 첫 호출 hang 은 막지 못한다.

```
const res = await fetch(url, { cache: "no-store" });
    const json = await res.json();
```

수정 힌트: `fetch(url, { cache: "no-store", signal: AbortSignal.timeout(6000) })` 로 타임아웃을 걸고, abort 시 catch 가 목업 폴백을 반환하게 한다(weather route 와 동일 패턴).

MEDIUM [3] 정확성·로직 #6 · apps/worker-watch-poc/Sources/VitalTransmitter.swift:172-180

VitalTransmitter pending 윈도는 타이머가 없어 다음 샘플 도착 시에만 flush

왜 문제: flushInterval(30초) 경과 판정과 실제 전송이 오로지 '다음 enqueuePending 호출'에 의존한다. 즉 타이머 기반 flush 가 없다. 워크아웃 세션이 살아있어도 HR 샘플 유입이 멈추면(예: 워치 착용 접촉 끊김, 센서 일시 정지) 마지막으로 모인 pending 배열이 전송되지 않고 무기한 머문다. flushPending() 은 stop() 시에만 호출되므로, 정지하지 않은 채 샘플 스트림만 끊긴 구간에서는 최신 생체데이터가 대시보드에 도달하지 않아 모니터링 연속성이 깨진다. 또한 마지막 윈도(< 30초)는 후속 샘플이 임계를 넘겨줄 때까지 항상 지연된다.

```
private func enqueuePending(_ payload: [String: Any]) {
    pending.append(payload)
    let now = Date()
    guard let last = lastFlushAt else {
        lastFlushAt = now   // 첫 샘플이 윈도 시작점 — 다음 flushInterval 후 전송.
        return
    }
    if now.timeIntervalSince(last) >= flushInterval { flush() }
}
```

수정 힌트: 윈도 시작 시 flushInterval 후 강제 flush 하는 타이머(Timer/DispatchSourceTimer)를 가동하고, 전송 성공 시 타이머를 재설정한다. 샘플 유입 의존 플러시를 보조 수단으로만 둔다.

LOW [3] 정확성·로직 #16 · apps/dashboard-web/app/api/mock/vital/route.ts:49-65, 76-92

배열 POST 처리 시 개별 addSample 의 ZodError 가 전체 요청을 400 으로 만들어 정상 샘플까지 유실

왜 문제: 워치 배칭 배열에서 한 항목이 스키마 검증에 실패하면 addSample(VitalSampleSchema.parse) 이 throw 하고 for 루프가 즉시 중단되어 catch 블록의 400 응답으로 빠진다. 이미 처리된 앞쪽 정상 샘플은 DB 에 들어갔지만 클라이언트는 400(실패)로 받아 backlog 를 재전송하게 되고(중복 수신) 뒤쪽 정상 샘플은 통째로 유실된다. 부분 실패가 전체 배치를 깨뜨리는 all-or-nothing 인데 응답 코드는 부분 성공 상태를 반영하지 못한다.

수정 힌트: 배열 루프에서 항목별 addSample 을 try/catch 로 감싸 검증 실패 항목만 dropped 로 집계하고 나머지는 계속 처리하도록 한다(현재 reserved 워커는 null 반환으로 이미 부분 처리됨 → 검증 실패도 동일하게 격리).

LOW [3] 정확성·로직 #17 · apps/dashboard-web/app/api/mock/history/route.ts:19-22

history GET 의 days 파라미터 NaN/빈값이 조용히 기본 7일로 폴백되어 잘못된 입력을 분간 못 함

왜 문제: days 가 없으면 Number(null)=0, 잘못된 문자열이면 NaN 이 되는데 둘 다 ALLOWED_RANGES 에 없어 조용히 7로 폴백한다. 발표 데모 mock 경로라 영향은 제한적이지만, workerId/metric 누락은 400 으로 거부하면서 range 만 silent default 라 일관성이 없고, 클라이언트의 days 오타가 의도와 다른 7일 차트로 표시되어 디버깅을 어렵게 한다.

수정 힌트: days 가 제공됐는데 ALLOWED_RANGES 에 없으면 400 으로 거부하거나, 최소한 기본값 적용 사실을 응답에 명시한다(워커/메트릭 검증과 동일한 엄격도).

LOW [3] 정확성·로직 #18 · apps/dashboard-web/app/api/admin/fall-label/route.ts:44-49

fall-label note 길이 무제한 — DB 무한 길이 문자열 적재 가능

왜 문제: label 은 ALLOWED set 으로 엄격 검증하지만 note 는 typeof 검사만 하고 길이 상한이 없다(아래 db.setFallLabel 도 trim 만 하고 cap 없음). 인증은 되어 있으나, 관리자 실수/스크립트가 매우 큰 문자열을 보내면 그대로 DB 에 적재된다. AED inspector(40자 slice)·alerts ackBy(40자 slice)·login route 와 달리 일관성 없는 비제한. 영향은 제한적(인증 필요)이라 LOW.

수정 힌트: note 를 .slice(0, 500) 등으로 상한 적용해 다른 입력 필드(inspector/ackBy 40자)와 일관성 유지.

LOW [3] 정확성·로직 #19 · apps/dashboard-web/app/lib/typhoon.ts:123-124

태풍정보 fetch 에 타임아웃 없음 — 업스트림 hang 시 요청 무한 대기

왜 문제: KMA 특보와 동일하게 data.go.kr TyphoonInfoService 호출에 타임아웃이 없다. 게이트웨이가 응답을 지연/매달리면 `fetch`·`res.text()` 가 무한 pending 되어 /api/typhoon 이 응답을 못 주고, 이를 폴링하는 대시보드 캐러셀 페이지가 멈춘다. kosha.ts 는 `AbortSignal.timeout(8000)` 을 쓰는데 이 파일만 빠져 있어 일관성도 깨진다.

수정 힌트: `fetch(url, { cache: "no-store", signal: AbortSignal.timeout(8000) })` 추가 → abort 시 기존 catch 가 mockTyphoon 폴백을 반환하도록.

LOW [3] 정확성·로직 #20 · apps/dashboard-web/app/api/radar/route.ts:20-22

레이더 프록시 fetch 에 타임아웃 없음 — RainViewer hang 시 요청 무한 대기

왜 문제: RainViewer 메타 호출에 타임아웃이 없어 업스트림이 응답을 끊지 않고 매달리면 /api/radar 가 무한 대기한다. 2분 캐시가 있어 빈도는 낮지만 캐시 만료 직후 hang 시 레이더 패널이 멈춘다. weather route 의 타임아웃 패턴과 불일치.

수정 힌트: `signal: AbortSignal.timeout(6000)` 추가. abort 시 기존 catch 가 502 로 강등 → 클라이언트가 직전 프레임 유지.

LOW [3] 정확성·로직 #21 · apps/dashboard-web/app/lib/youtube.ts:131-133

YouTube 검색 fetch 에 타임아웃 없음 — googleapis hang 시 특보영상 호출 대기

왜 문제: `searchOnce` 의 googleapis 호출에 타임아웃이 없다. 채널 화이트리스트(id 모드)에서는 `Promise.all(channels.map(...))` 로 여러 채널을 병렬 호출하므로 한 채널이라도 hang 하면 전체 검색이 끝나지 않고 /api/youtube-warning-videos 가 매달린다. 3시간 캐시가 있어 빈도는 낮지만 특보 발효 직후 첫 검색에서 hang 가능.

수정 힌트: `fetch(url, { cache: "no-store", signal: AbortSignal.timeout(6000) })` + 비-ok/abort 시 `return []`(이미 비-ok 강등 패턴 있음).

LOW [3] 정확성·로직 #22 · apps/dashboard-web/app/lib/youtube.ts:162-173

YouTube id 모드 검색이 Promise.all 사용 — 한 채널 실패로 전체 결과 손실

왜 문제: `searchOnce` 는 비-ok 시 `[]` 를 반환하지만 fetch 자체가 reject(네트워크 오류·타임아웃 추가 시)하면 `Promise.all` 이 즉시 reject → 바깥 try/catch 가 잡아 videos 를 통째로 `[]` 로 만든다. 정상 응답한 다른 채널 결과까지 버려져 특보영상 패널이 비어 보인다. `Promise.allSettled` 가 부분 성공을 보존한다.

수정 힌트: `Promise.allSettled` 로 바꿔 fulfilled 결과만 flat 처리(채널별 실패 격리).

LOW [3] 정확성·로직 #23 · apps/dashboard-web/app/components/DemoDashboard.tsx:639-642

load() 가 await response.json() 이후 active 재확인 없이 setData 호출 (언마운트/재폴 경합)

왜 문제: active 플래그를 response.ok 직후엔 확인하지만, response.json() 의 비동기 파싱이 끝난 뒤에는 재확인하지 않고 setData 한다. 2초 간격 폴링이라 직전 요청의 json() 파싱이 지연되면 cleanup(active=false) 이후에도 setData 가 실행될 수 있고, 또 늦게 도착한 응답이 최신 응답을 덮어쓰는 out-of-order 갱신이 가능하다. 프로토타입에선 영향 작지만 상용화 시 stale 데이터로 카드가 잠깐 되돌아갈 수 있다.

수정 힌트: const json = await response.json(); if (!active) return; setData(json); 형태로 await 후 active 를 재확인한다(또는 AbortController 로 in-flight 요청 취소).

LOW [3] 정확성·로직 #24 · apps/dashboard-web/app/components/DemoDashboard.tsx:687-708

saveJobTitle 성공/실패 분기에서 try-catch 의 catch 누락 — fetch 거부 시 unhandled rejection

왜 문제: try/finally 만 있고 catch 가 없다. fetch 가 네트워크 오류로 reject 되면(서버 다운·터널 만료 — 이 프로젝트는 세션마다 만료되는 quick tunnel 사용) 예외가 try 밖으로 전파되어 saveJobTitle 호출부(onSave 핸들러)에서 unhandled rejection 이 된다. finally 가 모달은 닫지만 낙관적 갱신은 안 되고 에러는 삼켜지지도 잡히지도 않는다.

수정 힌트: try 블록에 catch (e) { console.error(...) } 를 추가하거나, 호출부에서 await 한 Promise 를 .catch 처리한다. AlertHistoryModal.load(1969)·ack(1994) 도 동일 패턴(catch 없는 try/finally·무가드 fetch)이라 함께 점검.

LOW [3] 정확성·로직 #25 · apps/dashboard-web/app/components/CctvQuadView.tsx:336, 403

DetectionStrip·FallDetectionStrip 가 res.ok 미확인 후 .json() 호출 (PpeStrip 과 불일치)

왜 문제: 백엔드가 4xx/5xx 를 HTML/빈 본문으로 응답하면 .json() 이 throw 한다. 여기서는 try/catch 로 삼켜져(catch 블록 주석 '서버 미연결 — 다음 폴에서 재시도') 크래시는 없으나, PpeDetectionStrip 만 res.ok 를 검사하고 두 스트립은 검사하지 않아 일관성이 깨졌다. 에러 응답 시 throw→catch 경로로 stats 가 갱신되지 않은 채 stale 표시가 유지된다(낙상 0건 등으로 오인 가능).

수정 힌트: PpeDetectionStrip 과 동일하게 const res = await fetch(...); if (!res.ok) return; 패턴으로 통일한다.

LOW [3] 정확성·로직 #26 · apps/dashboard-web/app/components/WeatherStrip.tsx:108-138

WeatherStrip 날씨 fetch가 workerCount 변경에 재실행 안 됨 (useEffect 의존성 누락)

왜 문제: load() 클로저가 workerCount 를 URL 에 넣지만 useEffect 의존성은 [lat, lon] 뿐이다. 작업자 수가 바뀌어도(슬롯 등록/해제) 이펙트가 재실행되지 않고 기존 클로저가 캡처한 옛 workerCount 로 5분 간격 폴링이 계속된다. /api/weather 의 workers 파라미터는 weather_log 누적 메타(작업자 수)에 쓰이므로 잘못된 작업자 수가 영속 이력에 기록될 수 있다. lat/lon 이 바뀌면 따라 갱신되긴 하나 위치 고정 상태로 인원만 변하면 stale.

수정 힌트: 의존성 배열에 workerCount 추가하거나, weather_log 의 정확성이 중요치 않으면 URL 에서 workers 를 제거해 의도를 명확히 한다.

LOW [3] 정확성·로직 #27 · apps/dashboard-web/app/components/WeatherStrip.tsx:114-126

WeatherStrip 갱신 실패 시 setData 미갱신으로 stale 데이터가 무한 표시될 수 있음

왜 문제: 갱신 실패 시 error 만 set 하고 data 는 그대로 두는 graceful degrade 설계인데(주석 285-286), 화면 렌더 분기(287, 295)는 data 가 있으면 항상 마지막 값을 그린다. 업스트림이 장시간 다운되면 사용자에게 '몇 시간 전 값'이 실시간처럼 계속 보이고 시각적 stale 표시가 없다(updatedAt 만 옛 시각). 24/7 현장 모니터링에서 오래된 환경값을 신뢰하면 위험.

수정 힌트: error && data 동시 상태일 때 '갱신 지연/마지막 N분 전 값' 배지를 노출하거나, fetchedAt 이 일정 시간(예: 30분) 지나면 stale 경고를 표시한다.

LOW [3] 정확성·로직 #28 · apps/dashboard-web/app/components/SosAlertModal.tsx:35-41

SosAlertModal estimateCoreTempC(NaN skinTempC) 경로에서 NaN 비교가 normal 로 떨어질 수 있음

왜 문제: skinTempC 가 undefined 가 아닌 NaN(잘못된 페이로드/파싱)일 때 estimateCoreTempC(NaN)=NaN 이고 모든 >= 비교가 false 라 'normal' 로 분류된다. SOS/생체경보 모달에서 NaN 체온이 '정상(녹색)'으로 표시되면 false-OK. hrSeverity/spo2Severity 도 동일 패턴이나 체온은 estimate 거쳐 NaN 전파 가능성이 더 크다. 실수신 페이로드는 Zod 검증을 거치므로 프로토타입에선 가능성 낮지만 상용화 시 위험.

수정 힌트: Number.isNaN(core) 또는 Number.isFinite(skinTempC) 가드를 추가해 NaN 일 때 'idle'/'--' 로 처리한다.

LOW [3] 정확성·로직 #29 · apps/dashboard-web/app/components/SafetySignalBoard.tsx:156

SafetySignalBoard 의 KMA 응답 severity 를 검증 없이 LampSev 로 신뢰

왜 문제: weather 는 fetch JSON 을 KmaWarning 으로 단언(`as`)만 했을 뿐 런타임 검증이 없다. /api/kma-warning 이 오류 객체(`{error:...}`)나 severity 가 빠진/예상 외 문자열을 반환하면 weather.severity 가 STYLE 의 키에 없어 STYLE[weatherSev] 가 undefined 가 되고 165·205·211 라인에서 `STYLE[weatherSev].label`/`.dot` 접근 시 'Cannot read properties of undefined' 크래시로 신호등 보드 전체가 깨진다. 정상 응답일 땐 문제없으나 업스트림이 흔들리는 24/7 운영에선 방어가 필요(다른 파일 WeatherStrip 는 stale 폴백을 두는 반면 여기엔 없음).

수정 힌트: weather.severity 를 STYLE 키 집합으로 좁히는 가드(예: severity in STYLE ? severity : 'idle')를 두거나 KmaWarning 을 zod 등으로 파싱 후 사용.

LOW [3] 정확성·로직 #30 · apps/dashboard-web/app/components/WeatherRuleModal.tsx:213-216

HourlyTempSection 의 min/max 가 빈 배열에서 ±Infinity 가 되는 미방어

왜 문제: hourly 가 빈 배열이면 Math.min()/Math.max() 가 Infinity/-Infinity 를 반환하고 span 은 -Infinity(|| 1 로도 0 이 아니라 truthy 라 그대로 사용)가 되어 yAt 계산이 NaN 으로 오염된 SVG path 가 그려진다. 현재 호출부(83라인)는 `forecast.hourly && forecast.hourly.length > 0` 로 가드해 실제 도달은 안 하지만, 컴포넌트 자체가 빈 입력에 대한 방어가 없어 재사용 시 깨진다.

수정 힌트: 컴포넌트 진입부에서 hourly.length === 0 이면 null 반환하거나, min/max 에 기본값 폴백을 둔다.

LOW [3] 정확성·로직 #31 · apps/worker-phone/lib/main.dart:105-109

initState 비동기 초기화 경쟁 — _tryAutoStart 가 _loadReceiverUrl 완료 전에 실행될 수 있음

왜 문제: _loadReceiverUrl() 가 await 없이 호출되고, 그 안의 _storage.read(...) 가 비동기다. _tryAutoStart 는 microtask 로 즉시 큐잉되는데 _tryAutoStart 는 `_receiverUrl == null || _receiverUrl!.isEmpty` 면 그냥 return 한다. microtask 가 _loadReceiverUrl 의 storage read 완료(setState 로 _receiverUrl 채워짐)보다 먼저 실행되면 _receiverUrl 이 아직 null 이라 자동 시작이 조용히 건너뛰어진다. 주석은 'URL/프로파일 로딩 완료 후'라 의도하나, 실제로는 완료를 기다리지 않는다. 다만 _initConnectivityWatch 의 online 전이 시 _tryAutoStart 재호출 경로가 있어 영구 실패는 아니다.

수정 힌트: _loadReceiverUrl()/_loadProfile() 의 Future 를 await 한 뒤 _tryAutoStart 를 호출하도록 단일 async 초기화 함수로 묶어 순서를 보장한다(예: Future.microtask 대신 `await _loadReceiverUrl(); ... await _tryAutoStart();` 체인).

LOW [3] 정확성·로직 #32 · packages/analyzers/src/spo2.ts:33-57

SpO_2 caution이 단일 샘플로도 '30s 지속' 이벤트를 발화(sustain 미강제)

왜 문제: window 가 시간 구간 안의 샘플만 모으지만 최소 개수/실제 시간 폭 검증이 없다. recent 에 last 1개만 있고 spo2 가 88~91%면 window.length===1·allLow===true 가 되어 메시지는 '30s 지속'이라 표기하면서 실제로는 단일 측정 1건만으로 caution 을 발화한다. 지속(sustain) 의도가 하한 경계에서 강제되지 않아 false-positive 위험. 현재 detectSpo2Low 는 프로덕션 호출처가 없어(테스트만) 즉시 영향은 없지만, 백엔드 Lambda hazard-detect 에 연결되는 순간 단발 노이즈가 경보로 승격된다.

수정 힌트: window 가 실제로 cautionSustainSec 폭을 덮는지(최소 2개 이상 또는 첫 샘플 ts 가 earliestAllowed 근처) 확인하는 가드 추가 — 예: window.length < 2 또는 (last.ts - window[0].ts) < cautionSustainSec*1000 이면 caution 미발화.

LOW [3] 정확성·로직 #33 · packages/analyzers/src/heart-rate.ts:19-36

심박 이상이 단일 샘플로도 '5s 지속' 이벤트를 발화(sustain 미강제)

왜 문제: spo2.ts 와 동일 패턴. recent 에 last 1건만 있고 heartRate>=130(또는 <=40)이면 window.length===1 로 allHigh/allLow 가 충족되어 메시지엔 '5s 지속'이라 쓰면서 단일 샘플만으로 hr_abnormal warning 을 만든다. 지속성이 하한 경계에서 검증되지 않는다. detectHeartRateAbnormal 도 현재 프로덕션 호출처 0(테스트만)이라 즉시 영향은 없으나 백엔드 연결 시 일시적 스파이크가 경보로 승격될 수 있다.

수정 힌트: sustain 을 실제로 강제 — window 가 sustainSec 폭을 덮는지(window.length>=2 또는 last.ts - window[0].ts >= sustainSec*1000) 확인 후에만 트리거.

LOW [3] 정확성·로직 #34 · scripts/mock-publish.mjs:43-52

mock-publish.mjs 초기 publishBatch 의 await 에 에러 핸들링 부재(unhandled rejection)

왜 문제: setInterval 안의 반복 호출은 .catch 로 보호되지만(49행), 최초 1회 호출인 top-level await publishBatch()(43행)는 try/catch 없이 던진다. dev 서버가 아직 안 떠 있거나 fetch 가 실패하면 publish() 가 throw → 모듈 평가 단계에서 unhandled rejection 으로 프로세스가 즉시 종료되어 5초 주기 루프 자체가 시작되지 못한다. dev 전용 mock 발행 스크립트라 운영 영향은 없으나, 서버 기동 전 실행 시 조용히 죽는 UX 결함.

수정 힌트: 초기 호출도 .catch 로 감싸거나 try/catch 로 로그 후 진행 — 예: await publishBatch().catch((e) => console.error('[mock] initial publish failed', e));

[4] 성능 — 10건

MEDIUM [4] 성능 #7 · apps/dashboard-web/app/components/DemoDashboard.tsx:712-719

1초 now 틱이 allLatest→latestMap→pendingWorkers→fieldCoords 전체 useMemo 체인을 매초 재계산

왜 문제: allLatest 의 deps 에 매초 바뀌는 now 가 들어가 있어, signalLevel 표시 목적의 1초 틱(line 655-658 setNow)마다 allLatest 가 새 배열로 재생성된다. 이는 latestMap(721), pendingWorkers(801), fieldCoords(809), workerSignal(920) 등 latestMap/allLatest 에 의존하는 모든 useMemo 를 매초 무효화시키고, 그 결과 전체 작업자 표(WorkerTableRow N개 + overflow 시 2N개)와 SafetySignalBoard 가 1초마다 재렌더된다. now 가 가상 작업자의 ts 표시용으로만 쓰이는데도(렌더 표시상 '수신 중'처럼 보이기 위함) 데이터가 안 변해도 매초 전체 카드 트리가 다시 그려진다.

```
const allLatest = useMemo(() => {
    const realIds = new Set(data.latest.map((s) => s.workerId));
    const nowIso = new Date(now).toISOString();
    const virtuals: VitalSample[] = VIRTUAL_WORKER_TEMPLATES.filter(
      (v) => !realIds.has(v.workerId),
    ).map((v) => ({ ...v, ts: nowIso }));
    return [...data.latest, ...virtuals];
  }, [data.latest, now]);
```

수정 힌트: now 의존을 allLatest 에서 분리한다. 가상 작업자의 ts 는 mount/data 변경 시 한 번만 주입하거나, signalLevel 용 now 와 별개로 가상 ts 를 고정값(또는 더 긴 주기)으로 두어 latestMap/pendingWorkers/fieldCoords 가 data.latest 변경 시에만 재계산되게 한다.

LOW [4] 성능 #35 · apps/dashboard-web/app/api/mock/store.ts:56-58, 82-85

addSample 핫경로에서 mergeLatestByWorker 를 매 샘플 2회 호출(각각 ring 500 전체 filter+merge)

왜 문제: 모든 POST 수신마다(폰 5초/워치 30초 배칭, 워치 배열은 샘플 1건마다) 같은 워커 샘플을 ring(최대 500)에서 두 번 필터링하고 mergeLatestByWorker(Map 빌드+정렬)를 두 번 돌린다. 단일 워커 데모에선 무시할 수준이나, 상용화 시 작업자 수·수신 빈도가 늘면 매 수신이 O(samples) × 2 가 되어 ingest 처리량을 갉아먹는다. prevMerged 는 추가 직전 상태, curMerged 는 추가 직후 상태가 필요하므로 두 번 도는 것 자체는 의미가 있으나, 워커별 최신 병합값은 단순 누적으로 O(1) 유지 가능한 데이터다.

수정 힌트: 워커별 최신 병합 결과를 별도 Map 에 증분 유지(addSample 에서 prev 를 읽고 새 샘플로 갱신)하거나, 최소한 같은 worker 필터 결과를 한 번만 만들어 prev/cur 두 병합에 재사용해 filter+정렬 중복을 제거한다.

LOW [4] 성능 #36 · apps/dashboard-web/lib/admin-auth.ts:93-107

세션 토큰 Map 만료 항목 미수거 — 장기 가동 시 메모리 누수

왜 문제: sessions Map 은 createSession 마다 항목이 추가되지만, 만료 항목은 '그 토큰을 다시 validateSession 으로 조회할 때만' lazily 삭제된다. 로그인 성공(또는 재로그인)이 반복되면 이전 토큰들은 다시 조회되지 않으므로 영원히 Map 에 잔존한다 → 프로세스 장기 가동(상용 24/7) 시 만료된 토큰이 무한 누적되는 메모리 누수. 프로토타입(빈번한 dev 재시작)에선 영향 제한적이나 상용화 시 주기적 sweep 이 필요.

수정 힌트: 주기적(setInterval 또는 createSession N회마다) 만료 토큰 sweep 추가, 또는 sessions 크기 상한 + LRU eviction.

LOW [4] 성능 #37 · apps/dashboard-web/app/components/DemoDashboard.tsx:1376-1378

computeStressIndex 를 행마다 렌더 중 인라인 호출(메모 없음) — 매초·매 폴마다 N(또는 2N)회 재계산

왜 문제: 각 작업자 행 렌더 시 computeStressIndex(로지스틱·z-score 등 비순수-비용 연산)를 useMemo 없이 직접 호출한다. 자동 모드 overflow 시 행이 2벌 복제(line 1361)되어 호출 횟수가 2N 으로 배가되고, 위 finding 처럼 now 틱·2초 폴마다 latestMap 이 새로 생겨 React.memo 도 못 걸리는 구조라 매초 모든 행에서 재계산된다. 수동 모드(1340)에도 동일 패턴. 작업자 수가 늘수록(SLOT_COUNT 100) 비용이 선형 증가한다.

수정 힌트: workerId→StressResult 를 latestMap/data.latest 기준 useMemo Map 으로 1회 계산해 행에 전달하거나, WorkerTableRow 를 React.memo 로 감싸고 sample 정체성을 안정화한다(위 allLatest now 분리와 병행).

LOW [4] 성능 #38 · apps/dashboard-web/app/components/DemoDashboard.tsx:646-651

load 폴링 effect 가 latestMap/data 변화와 무관히 cleanup 시 active 만 끄고 in-flight fetch 미취소

왜 문제: cleanup 이 interval 은 제거하지만 진행 중인 fetch 는 취소(AbortController)하지 않는다. 컴포넌트가 빠르게 마운트/언마운트되거나 effect 가 재실행될 때 dangling fetch 가 남아 응답 파싱 후 위 라인의 active 가드에만 의존한다. deps 가 [] 라 실질 1회 등록이지만, 누수 방지·요청 정리 관점에서 AbortController 가 없다.

수정 힌트: AbortController 를 생성해 fetch 에 signal 을 넘기고 cleanup 에서 abort() 한다. 표시상 영향은 작으나 상용화 시 명시적 요청 취소가 안전하다.

LOW [4] 성능 #39 · apps/dashboard-web/app/components/SiteSafetyTables.tsx:227, 756-774

AedAlertTable 마운트마다 12초 폴링이 모듈 전역 캐시를 덮어써 다중 인스턴스 상호 간섭

왜 문제: AedAlertTable 가 동일 페이지에 두 번 이상 마운트되면 각 인스턴스가 자기 setInterval(12초)로 동일한 모듈 전역 aedInspectionsCache 에 fetch 결과를 써넣어 중복 네트워크 호출 + 전역 가변 상태 경합이 생긴다. 현재는 단일 캐러셀 페이지라 영향이 제한적이지만, 전역 가변 캐시를 컴포넌트 effect 가 쓰는 패턴은 테스트·재사용성을 떨어뜨린다.

수정 힌트: 점검 캐시를 모듈 전역 let 대신 React state/context 또는 SWR 같은 공유 캐시로 끌어올려 인스턴스 독립성과 단일 폴링을 보장한다.

LOW [4] 성능 #40 · apps/dashboard-web/app/components/SiteSafetyTables.tsx:793, 857

AedAlertTable 렌더에서 행마다 aedStatus()를 두 번 호출(맵 본문 + 모달)

왜 문제: 행 맵에서 이미 aedStatus(r) 를 계산하는데 모달을 띄울 때 동일 row 에 대해 aedStatus(alertRow) 를 한 번 더 계산한다. aedStatus 는 aedIssues→배터리/패드/단말 3차원 + sort 를 매번 새로 수행하므로 매 리렌더(12초 폴링 setTick 포함)마다 중복 연산이 발생한다. 행 수가 적어 비용은 작지만 불필요한 재계산이다.

수정 힌트: alertRow 설정 시 status 를 함께 저장하거나 useMemo 로 alertRow 의 status 를 캐시해 중복 호출을 제거한다.

LOW [4] 성능 #41 · apps/dashboard-web/app/components/RadarRealtime.tsx:50-70

RadarRealtime SquareFit·useIsDark·ResizeObserver/Interval 가 모달과 인라인에서 중복 마운트되며 측정 ResizeObserver 다수 생성

왜 문제: 팝업(open)을 열면 인라인 RadarTiles 와 모달 RadarTiles 가 동시에 마운트되어 각각 SquareFit ResizeObserver + 베이스맵/레이더 타일 <img> 그리드(줌6 + 줌7) 가 둘 다 살아 있다(인라인이 언마운트되지 않음). 모달 표시 중 타일 이미지가 2배 로드·관찰되어 불필요한 네트워크/측정 비용이 든다. 메모리 누수는 아니지만(cleanup 정상) 동시 렌더 낭비.

수정 힌트: 팝업이 열려 있는 동안 인라인 RadarTiles 렌더를 일시 중단(open && return placeholder)하거나, 타일 그리드를 메모이즈해 재계산을 줄인다.

LOW [4] 성능 #42 · apps/dashboard-web/app/components/SafetySignalBoard.tsx:189-193

SafetySignalBoard 가 매 렌더 gasDetail()/aedDetail() 를 호출

왜 문제: 이 컴포넌트는 10분 주기 setInterval 외에도 부모(DemoDashboard) 의 잦은 폴링 리렌더에 함께 리렌더된다. 매 렌더마다 gasDetail()/aedDetail() 가 재실행되는데 이 함수들이 객체/배열을 새로 생성하면 불필요한 연산이 반복된다. 데모 규모에선 영향이 작지만 useMemo 로 감싸면 제거 가능한 낭비다.

수정 힌트: gas/aed 행을 useMemo 로 메모이즈하거나, 입력 의존성이 없으면 컴포넌트 외부 상수로 끌어올린다.

LOW [4] 성능 #43 · apps/dashboard-web/app/components/LiquidMetalButton.tsx:49-68

LiquidMetalButton 의 리플 제거 setTimeout 이 언마운트 시 정리되지 않음

왜 문제: 클릭 직후(리플 600ms 수명 내) 컴포넌트가 언마운트되면 — 예: 로그인 CTA 클릭으로 onLogin 이 LoginGate 의 authed=true 를 만들어 LoginScreen 트리가 즉시 사라지는 경우 — setTimeout 콜백이 언마운트된 컴포넌트의 setRipples 를 호출한다. ref/timer id 를 추적·정리하지 않아 정확히 이 로그인 흐름에서 발생할 수 있다. 크래시는 아니지만 (React 18+는 경고 안 함) 댕글링 타이머·불필요한 상태 업데이트다.

수정 힌트: 타이머 id 들을 useRef 배열/Set 에 모아 두고 컴포넌트 언마운트 useEffect cleanup 에서 clearTimeout 으로 모두 정리한다.

[5] 아키텍처 — 0건 ✓ 클린

analyzers 순수성(I/O 금지)·backend↔dashboard 분리·순환 의존 무위반.

[6] 테스트 — 6건

MEDIUM [6] 테스트 #8 · apps/dashboard-web/app/api/mock/store.ts:42-99

store.ts addSample 수신 게이트 본체(파싱·알림 기록·생체 전이) 테스트 전무

왜 문제: store.test.ts 는 순수 함수 mergeLatestByWorker/isReservedTestWorker 만 검증하고 핵심 수신 게이트 addSample 은 한 줄도 테스트하지 않는다 (grep 결과 어떤 test 파일도 addSample/recordAlert 미참조). addSample 은 Zod 파싱·reserved 차단·in-memory ring slice(-500)·생체 위험 정상→위험 전이당 1건 기록(prevMerged vs curMerged)·SOS/낙상 알림 기록을 모두 담당하는 가장 복잡한 분기. 특히 '전이당 1건 + 지속 시 dedup'(세션32 핵심 결정)은 prevMerged/curMerged 병합 비교에 의존하는데, 회귀 시 잠금 null 깜빡임마다 vital 알림이 중복 기록되거나(false alert 폭주) 전이를 놓칠(미기록) 수 있다. 현재는 Playwright 런타임 검증에만 의존해 단위 회귀 안전망이 없다.

```
export function addSample(input: unknown) {
  const normalized = ... "hr" in input ? { ...input, heartRate: ... } : input;
  const sample = VitalSampleSchema.parse(normalized);
  if (isReservedTestWorker(sample.workerId)) return null;
  ...
  if (!vitalDanger(prevMerged) && vitalDanger(curMerged)) { recordAlert({ ... type: "vital", ts: curMerged?.ts ?? sample.ts
}); }
```

수정 힌트: recordAlert/persistSample 를 주입 가능하게 하거나 mock 한 뒤, addSample 에 대해 (a) hr 별칭이 heartRate 로 정규화되는지 (b) reserved workerId 가 null 반환 (c) 정상→danger 전이 1건만 vital 알림 기록 (d) 지속 danger 시 추가 기록 0건(dedup) (e) SOS/낙상 샘플은 vital 분기 건너뛰는지 테스트 추가.

MEDIUM [6] 테스트 #9 · packages/analyzers/src/heart-rate.ts:31-36

heart-rate 서맥(allLow ≤40bpm) 경로 미검증 — high 경로만 테스트

왜 문제: analyzers.test.ts 의 심박 테스트 2건(39-60행)은 모두 allHigh(≥130) 경로만 다룬다. 서맥(저심박 ≤40bpm) allLow 경로는 warning 이벤트를 생성하는 정식 위험 감지 분기인데 한 번도 실행되지 않아, lowBpm 임계나 every() 술어가 깨져도(예: 부등호 방향, ?? 폴백 값) 테스트가 통과한다. 저체온·실신성 서맥은 산업현장 핵심 위험이라 미감지 회귀가 치명적이다.

```
const allLow = window.every(
  (s) => (s.heartRate ?? Number.POSITIVE_INFINITY) <= THRESHOLDS.heartRate.lowBpm,
);
if (!allHigh && !allLow) return null;
```

수정 힌트: 최근 윈도가 전부 ≤40bpm(예: 38/35/40)인 케이스로 hr_abnormal warning 생성을, 일부만 낮은 케이스로 null 을 검증하는 테스트 추가.

MEDIUM [6] 테스트 #10 · packages/analyzers/src/spo2.ts:40-46

spo2 caution 윈도 비-지속(allLow=false) 및 빈/undefined 가드 미검증

왜 문제: analyzers.test.ts 의 SpO_2 테스트 2건(69-89행)은 즉시 danger(<88)와 30s 전부 저하 caution 의 양성 케이스만 다룬다. caution 윈도에 정상값(예: 95)이 섞여 allLow=false 가 되는 비-트리거 경로, recent.length===0 빈 입력 가드, last.spo2===undefined 가드는 모두 미검증. allLow every() 술어가 회귀하면 30s 미지속 일시 저하를 caution 으로 오경보(false positive)하거나 그 반대가 되는데 테스트가 못 잡는다.

```
const window = recent.filter(
  (s) => s.spo2 !== undefined && new Date(s.ts) >= earliestAllowed,
);
if (window.length === 0) return null;
const allLow = window.every((s) => (s.spo2 ?? 100) < THRESHOLDS.spo2.cautionPct);
if (!allLow) return null;
```

수정 힌트: 윈도에 정상 SpO_2 1건 섞은 케이스로 null(비-트리거), recent:[] 와 spo2 미동봉 샘플로 null 가드를 각각 검증.

MEDIUM [6] 테스트 #11 · packages/analyzers/src/stress-index.ts:127-140

stress-index respiratoryRate 신호·개인 baseline 인자·light 활동 분류 미검증

왜 문제: stress-index.test.ts 5건은 rest/heavy 활동과 HRV·ExcessHR 신호만 다룬다. (a) respiratoryRate 신호3(zRr) 경로 (b) computeStressIndex 의 두 번째 인자 personal baseline 오버라이드 (c) light 활동 분류(walkingHeartRate 기대값 경로) (d) index 가 정확히 40/70 경계일 때 caution/danger 레벨 매핑 — 모두 미검증. 신호 결합 가중치(COMBINE)나 baseline 폴백 로직이 회귀해도 테스트가 통과한다. 스트레스 지수는 카드에 노출되는 위험 추정값이라 산식 회귀가 오판으로 직결된다.

```
const zRr = inputs.respiratoryRate !== undefined
  ? (inputs.respiratoryRate - baseline.respiratory) / baseline.respiratory : 0;
...
const raw = aEff * clip(zHrv, 0, 3) + bEff * clip(zHr, 0, 0.5) * 6 + COMBINE.c * clip(zRr, 0, 0.5) * 6;
```

수정 힌트: respiratoryRate 동봉 샘플로 지수 상승 확인, 개인 baseline 전달 시 population 대비 다른 결과, light 분류(activeEnergyKcal 4·steps 100 등) 케이스, level 경계 입력을 각각 추가.

LOW [6] 테스트 #44 · packages/analyzers/src/fall.ts:14-16

fall 입력 누락 가드(accel/still undefined) 미검증

왜 문제: analyzers.test.ts 의 낙상 테스트 2건(13-37행)은 둘 다 accel·still 이 채워진 샘플만 사용한다. 가속도/무동작 필드가 누락된 샘플(일반 vital 폴 샘플 — 대부분의 수신 샘플이 여기 해당)에서 null 반환하는 가드는 미검증. 이 가드가 깨지면 accel/still undefined 에 대해 비교 연산이 NaN/예외로 흐를 수 있는데, detectFall 은 일반 폴 샘플마다 호출될 수 있어 영향 범위가 넓다.

수정 힌트: accelMagnitudeG 만 있고 motionStillSec 없는(또는 둘 다 없는) 샘플로 detectFall 이 null 반환하는지 테스트 추가.

LOW [6] 테스트 #45 · packages/schemas/src/vital-stream.ts:17-26

VitalSampleSchema 이벤트·ML 필드 및 skinTempC 하한 검증 누락

왜 문제: vital-stream.test.ts 는 workerId/ts/heartRate/spo2/skinTempC 상한/위경도만 검증한다. 수신 게이트가 의존하는 이벤트 플래그(manualSos/fallDetected — store.ts addSample 이 이 boolean 으로 알림 기록 분기)와 ML 수집 필드(fallDecision enum, peakRotationRadS 범위, fallCandidate)는 한 건도 검증되지 않는다. 잘못된 타입(문자열 boolean·미정의 enum 값)이 통과되면 addSample 의 if(sample.manualSos) 분기와 fall ML CSV 내보내기가 오동작한다. skinTempC 도 상한(50 거부)만 있고 하한(20 미만 거부)은 미검증.

수정 힌트: manualSos:"yes"(string) 거부, fallDecision:"bogus" 거부·"fall" 통과, peakRotationRadS:51 거부, skinTempC:19 거부 케이스를 schema 테스트에 추가.

[7] 의존성·설정 — 2건

LOW [7] 의존성·설정 #46 · apps/dashboard-web/app/ai-cctv/[...path]/route.ts:6

AI_CCTV_ORIGIN env consumed by CCTV proxy route but undocumented in .env.example

왜 문제: The same-origin CCTV streaming proxy reads process.env.AI_CCTV_ORIGIN to choose the upstream backend host, but .env.example documents none of it. An operator deploying the dashboard on a host where the CCTV backend is not on localhost:8000 has no documented way to discover this knob — it silently falls back to localhost and the proxy returns errors. .env.example is supposed to be the single source of truth for configurable env, and this key is missing.

수정 힌트: Add an AI_CCTV_ORIGIN entry (with the localhost:8000 default and purpose comment) to apps/dashboard-web/.env.example alongside the other documented keys.

LOW [7] 의존성·설정 #47 · apps/dashboard-web/app/components/CctvQuadView.tsx:10

NEXT_PUBLIC_AI_CCTV_API env consumed by CctvQuadView but undocumented in .env.example

왜 문제: The CCTV tile component reads the public env NEXT_PUBLIC_AI_CCTV_API to point the browser at the MJPEG stream/stats API (CLAUDE.md documents setting it to /ai-cctv for team-share via the proxy), yet .env.example lists none of the CCTV env vars. A new clone has no documented record that this var exists or what value enables team sharing, so the proxy path can be missed and CCTV silently breaks for shared viewers (the exact failure mode CLAUDE.md session36 spent time diagnosing).

수정 힌트: Document NEXT_PUBLIC_AI_CCTV_API in apps/dashboard-web/.env.example with the /ai-cctv (same-origin proxy) recommended value and the localhost:8000 default, mirroring the AI_CCTV_ORIGIN entry.

[8] Dead code — 8건

LOW [8] Dead code #48 · apps/dashboard-web/app/components/SiteSafetyTables.tsx:173-175

GasDetectorTable 행의 ring-red-500/0 은 항상 투명(시각 효과 0 데드 클래스)

왜 문제: sev 가 normal 이 아닐 때 ring-1 ring-inset ring-red-500/0 을 붙이지만 /0 은 알파 0 = 완전 투명이라 화면상 아무 링도 보이지 않는다. 위험/주의 행을 강조하려는 의도로 보이나 실제로 렌더되는 시각 효과가 전혀 없는 죽은 스타일이다(작업자 표는 ring-red-500/50 등 가시 알파 사용).

수정 힌트: 강조가 의도면 ring-red-500/50 같은 가시 알파로 바꾸고, 의도가 아니면 조건부 클래스 자체를 제거한다.

LOW [8] Dead code #49 · apps/dashboard-web/app/components/CctvQuadView.tsx:23, 151, 166

Cam.aiScale 필드와 scale transform 경로가 데이터 없이 도달 불가(dead)

왜 문제: CAMERAS 의 8개 카메라 정의 중 aiScale 을 지정한 항목이 0개라 cam.aiScale 은 항상 undefined → AiLiveScreen 의 scale prop 도 항상 undefined → 151행의 transform: scale(...) 분기는 실행될 수 없는 코드다. 세션36 노트(CSS scale 로 화각 줌아웃 불가 → 되돌림)와 일치하는 잔여 배선으로, 동작에는 영향 없으나 도달 불가 코드 + 미사용 옵션 필드가 남아 있다.

수정 힌트: aiScale 필드와 AiLiveScreen/CameraScreen 의 scale prop·151행 transform 분기를 함께 제거하거나, 실제로 줌아웃이 필요하면 CAMERAS 항목에 aiScale 값을 지정해 경로를 활성화한다.

LOW [8] Dead code #50 · apps/dashboard-web/app/components/heroui-combo-box.tsx:724-740

heroui-combo-box 미사용 export 다수 (Avatar/Surface/FieldError/Form/Button/Label/Description/ChevronsExpandVertical/AvatarFallback/AvatarImage)

왜 문제: 유일한 import 처(LoginSiteSelect.tsx:7)는 ComboBox, Input, ListBox, Header, Separator 만 사용한다. Avatar/AvatarFallback/AvatarImage/Button/ChevronsExpandVertical/Description/FieldError/Form/Label/Surface 는 레포 전체에서 import 0건(grep 확인). 21st.dev 풀포트의 잔여 컴포넌트로, 번들에 죽은 코드 + 관련 .combo-avatar/.button/.surface/.field-error CSS(192-198 등)도 함께 실린다.

수정 힌트: 현장 선택에 실제 필요한 export(ComboBox/Input/ListBox/Header/Separator)만 남기거나, 풀킷 유지가 의도면 파일 상단 주석에 '미사용 컴포넌트 포함' 명시.

LOW [8] Dead code #51 · apps/worker-phone/lib/services/vital_publisher.dart:56-85

connectMqtt / AwsIotConfig / MQTT 분기 — 호출처 0 (도달 불가 코드 경로)

왜 문제: grep 결과 connectMqtt 및 AwsIotConfig 는 lib/·test/ 어디에서도 호출되지 않는다(_client 는 항상 null → _sendOne 의 MQTT 분기 영구 도달 불가, HTTP fallback 만 활성). 코드 자체 주석(53-55)이 'AWS 결제 보류로 의도적 미연결, 삭제하지 않고 보존'이라 명시하므로 의도된 보존이나, mqtt_client 의존성(pubspec)·SecurityContext 코드 전체가 사실상 죽은 경로다. 프로토타입 스코프상 LOW.

수정 힌트: 발표 후 AWS 트랙 착수 전까지는 의도된 보존이므로 유지하되, 향후 빌드 크기/의존성 정리 시 mqtt_client 와 함께 별도 모듈로 분리하거나 제거 검토.

LOW [8] Dead code #52 · apps/worker-phone/lib/theme/app_tokens.dart:14

AppTokens.onAccentDeep 상수 미사용 (dead)

왜 문제: grep 결과 onAccentDeep 은 정의 파일 외 lib/·test/ 어디에서도 참조되지 않는다. accentDeep 버튼의 전경색은 main.dart 에서 Colors.white 를 직접 지정(line 997)하고 있어 이 토큰은 사용되지 않는 죽은 상수다. (bgBottom·ambientTempC 는 실제 사용 확인됨)

수정 힌트: 미사용 상수 제거하거나, 일관성을 위해 main.dart 의 accentDeep 버튼 foregroundColor 에 onAccentDeep 을 실제로 사용.

LOW [8] Dead code #53 · apps/worker-watch-poc/Sources/WorkoutManager.swift:30

WorkoutManager.hrUnit 미사용 프로퍼티

왜 문제: 선언되었으나 어디서도 참조되지 않는다(grep 결과 정의 1건 외 사용처 0). HR BPM 변환은 delegate 내부(221행)에서 별도의 로컬 let unit = HKUnit(from: "count/min") 로 다시 만들어 쓰므로 이 인스턴스 프로퍼티는 도달하지 않는 죽은 코드다.

수정 힌트: hrUnit 프로퍼티를 제거하거나, delegate 의 로컬 unit 재생성을 제거하고 이 프로퍼티를 사용해 중복을 없앤다.

LOW [8] Dead code #54 · apps/worker-watch-poc/Sources/ConnectionSettings.swift:47-49

ConnectionSettings.url 인스턴스 계산 프로퍼티 미사용

왜 문제: 트랜스미터는 static currentURL()(52행)만 사용하고, SettingsView 는 urlString(text 바인딩)만 쓴다. grep 결과 인스턴스 .url 호출처가 0건이라 도달하지 않는 죽은 코드다. static currentURL() 와 로직이 중복돼 향후 한쪽만 수정하면 drift 위험도 있다.

수정 힌트: 사용처가 없으면 인스턴스 url 프로퍼티를 제거한다.

LOW [8] Dead code #55 · apps/worker-watch-poc/Sources/FallDetector.swift:63

FallDetector.isAvailable 미사용 프로퍼티

왜 문제: 공개 계산 프로퍼티지만 프로젝트 전체에서 호출처가 0건이다(grep 으로 isDeviceMotionAvailable/isHealthDataAvailable 제외 후 isAvailable 참조 없음 확인). start() 가 내부적으로 motion.isDeviceMotionAvailable 를 직접 가드하므로 이 외부 노출 프로퍼티는 죽은 코드다.

수정 힌트: 외부에서 가용성 점검이 불필요하면 제거한다.

[9] 문서·일관성 — 5건

LOW [9] 문서·일관성 #56 · README.md:N/A (파일 부재)

담당 파일 README.md(루트)가 실제로 존재하지 않음

왜 문제: 검사 대상으로 지정된 루트 README.md 가 디스크에 없다. 프로젝트 루트 진입 문서가 부재해 신규 협업자/팀원이 저장소를 열었을 때 개요·실행법을 즉시 얻지 못한다(백엔드 README 는 backend/infrastructure 하위만 다룸). docs/checklist.md·progress.md 가 그 역할을 대신하지만 루트 관례상 README 부재는 onboarding 일관성 갭이다.

수정 힌트: 루트 README.md 를 생성해 CLAUDE.md/docs 로의 진입 링크(상태=progress.md, 할 일=checklist.md, 빌드=CLAUDE.md '빌드/테스트') 만 담은 얇은 인덱스로 두거나, 의도적으로 두지 않는다면 본 파일을 검사 대상 목록에서 제외하도록 워크플로 설정을 조정한다.

LOW [9] 문서·일관성 #57 · docs/checklist.md:3

checklist.md 진척률(D2~88%/D3~68%)이 CLAUDE.md 현재 상태(D2~92%/D3~78%)와 불일치

왜 문제: 같은 저장소의 두 주요 상태 문서가 서로 다른 진척률을 주장한다. CLAUDE.md Status 는 'D1 ~99% + D2 ~92% + D3 ~78%'(세션37 기준)인데 checklist.md 헤더는 세션30·D2~88%/D3~68%로 멈춰 있다. 또한 progress.md Quick Summary 는 세션37까지 갱신됐는데(line 15) checklist 헤더는 세션30, 본문 '다음 할 일'(line 21·124)은 세션9/27 내용에 머물러 7세션 stale 이다. 진척률은 발표 준비 의사결정의 핵심 지표라 불일치가 혼선을 준다.

수정 힌트: 진척률 수치를 단일 소스(CLAUDE.md Status 또는 progress.md 마일스톤 표)로 일원화하고 checklist.md 헤더는 그 소스를 링크 참조만 하도록 바꿔 동기화 부담을 제거한다.

LOW [9] 문서·일관성 #58 · docs/progress.md:495-496, 526-531

progress.md '현재 상태/마일스톤' 표가 동일 파일이 stale 이라 명시한 내용을 그대로 노출

왜 문제: progress.md 하단 두 상태 블록(line 495-507, 526-531)은 발표 마감 2026-06-04·블로커 'Galaxy Health 권한 대기'·'Step 2 권한 미수행 강력 추정'을 단정하지만, 같은 파일 line 530 은 Galaxy Health 가 세션32에 해결됐다고, line 531 은 '본 Snapshot 은 stale'이라고 스스로 반박한다. CLAUDE.md 도 'Step 2 권한 추정은 틀렸음(stale APK 원인)'으로 정정한 상태다. 자기모순 블록을 남겨두면 1분 안에 현황 파악이 목표인 본 파일의 신뢰도가 떨어진다.

수정 힌트: line 492-531 의 stale 상태 블록을 삭제하거나 명시적으로 '아카이브(시점 기록)' 헤더 아래로 옮기고, 마일스톤 진척률·블로커는 Quick Summary 한 곳에서만 유지한다.

LOW [9] 문서·일관성 #59 · docs/checklist.md:190-198

checklist.md D3 '대시보드 Alert/Flutter SOS UI' 항목이 모두 미완료([ ])이나 코드에 구현됨

왜 문제: D3 의 청각 알림·상태 색 동기화·Flutter SOS 버튼은 미완료로 표기됐으나 코드에 존재한다 — DemoDashboard.tsx:959-961 에 Web Audio 사이렌(`audioCtxRef`/`sirenRef`), main.dart:1042 에 'SOS 버튼 — 3초 길게 눌러야 발신', SosAlertModal.tsx, 그리고 checklist 자신의 D1 line 96-97 이 `overallStatus`/`SignalBars` 상태 색 동기화를 `[x]`로 이미 표기한다. 동일 기능이 D1 에선 완료, D3 에선 미완료로 상충한다(D3 섹션이 AWS HazardEvent 아키텍처 기준이라 부분 완화되나 대시보드/폰 UI 항목은 명백히 완료).

수정 힌트: D3 '대시보드 Alert'·'Flutter 폰 SOS UI' 중 실제 구현된 항목(청각 알림·상태 색 동기화·SOS 버튼·3초 게이지)을 `[x]`로 갱신하고, 미구현된 AWS Lambda HazardEvent 경로 항목만 `[ ]`로 분리 표기한다.

LOW [9] 문서·일관성 #60 · backend/infrastructure/README.md:23

backend README 가 drift 방지를 표방하면서 죽은 cloudflared URL 을 '발급' URL 로 박제

왜 문제: 같은 README line 3 은 '중복 status 블록 제거 — drift 방지'를 선언하며 상태를 progress.md 로 위임한다고 명시한다. 그런데 line 23 은 세션마다 회전·만료되는 quick tunnel URL `provisions-corpus-nav-discussed` 을 특정해 '발급'됐다고 서술한다. 이 URL 은 progress.md 의 어느 세션 active URL 과도 일치하지 않는 죽은 값으로, README 가 표방한 drift 방지 원칙을 스스로 위반한다.

수정 힌트: line 23 의 구체 URL 을 '`https://<무작위>.trycloudflare.com` (세션마다 회전, 현재 URL 은 progress.md 참조)' 형태의 플레이스홀더로 바꿔 만료되는 값이 README 에 박제되지 않게 한다.

3. 우선순위별 수정 권장 순서

Tier 0 — 공개 URL 공유·상용화 전 (보안·안전 핵심)

1. **#1 FallDetector 데이터 레이스** — 안전 핵심 + Swift 6 컴파일 차단. 상태 접근을 단일 직렬 큐/actor 로 격리.
2. **#2 AED 점검 무인증 POST + #3 CCTV 오픈 프록시** — 공개 cloudflared 공유 시 즉시 악용. 경로 allowlist·서명·인증 게이트.
3. **#4 vital alert 활동 게이트 토글** — 안전 알림이 위험을 가릴 수 있음(false-negative). 전이 판정은 안정 임계.

Tier 1 — 발표 전 권장 (24/7 안정성·false-OK)

1. fetch 타임아웃 일괄(#5·#19·#20·#21·#22) — AbortSignal.timeout + Promise.allSettled 패턴 통일.
2. #16 배열 부분실패 격리 + #28·#29·#30 NaN/undefined/Infinity 가드(false-OK 차단).
3. #8 핵심 수신 게이트(addSample) 테스트 — #4 같은 회귀 안전망.

Tier 2 — 상용화 트랙

1. 성능: #7·#37(재렌더)·#35(ingest 핫경로)·#36·#43(메모리 누수).
2. 분석기 테스트 보강: #9(서맥)·#10·#11·#32·#33·#44·#45.
3. 보안 하드닝: #13(IP 기준 lockout)·#14(스킴 분기)·#15(기본 자격증명 제거).
4. 위생: #46·#47(.env.example)·#48~#55(dead code)·#56~#60(문서 동기화).

검증 신뢰도 — 모든 발견은 적대적 검증자가 인용 file:line 을 직접 재확인했고 부정확/맥락누락 3건은 제거됨. 본 audit 는 코드를 수정하지 않음.